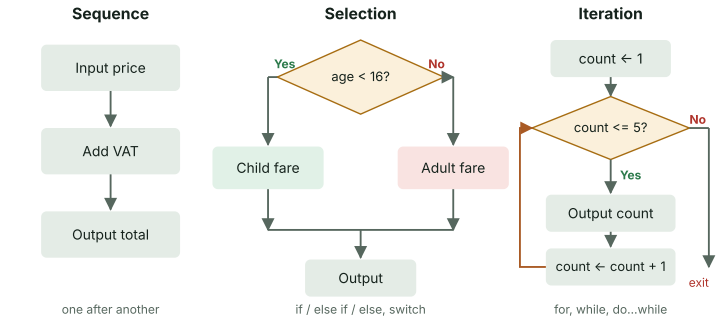
Program control: sequence, selection and iteration

AP20 FACT FILE

Specification: students should be able to

- Describe the fundamental programming concepts of sequence, selection and iteration
- Use if, if...else, else if, nested if and switch statements for selection, with conditions built from relational and Boolean operators
- Describe count-controlled iteration and use for loops, including loops that count up, down and in steps
- Describe condition-controlled iteration and use while (pre-condition) and do...while (post-condition) loops, including sentinel values
- Use nested loops and nested selection, and trace programs that combine all three constructs

Key words: sequence, selection, iteration, condition, if...else, else if, nested selection, switch, count-controlled, condition-controlled, for loop, loop counter, while loop, do...while loop, sentinel value, nested loop, infinite loop, trace table



The three control structures as flowcharts: sequence, selection with two branches, and iteration with a loop back

Overview: controlling the flow of a program

Every program, however large, is built from three control structures. Sequence means carrying out instructions one after another in the order they are written, so the position of each statement matters: a total must be calculated after the values it depends on have been input and before it is output. Selection lets a program choose between different paths depending on a condition that evaluates to true or false, for example charging a different fare for a child or adding a fee only for an overweight bag. Iteration repeats a block of statements, called the body of the loop, so the same code does not have to be written out many times. Iteration comes in two forms. Count-controlled loops repeat a number of times known before the loop starts, while condition-controlled loops repeat while, or until, a condition is met, so the number of repetitions does not need to be known in advance. These structures can be nested inside one another: a loop may contain decisions, and one loop may contain another. Being able to identify each structure in a given algorithm, choose the right one for a task, write it in C# and trace it by hand are all core skills tested in AS 1.

Selection with if, else if and nested if

An if statement evaluates a condition and runs its block only if the result is true; if it is false the block is skipped and execution continues after it. Adding else gives two mutually exclusive paths: exactly one of the two blocks will run, never both. When there are several possibilities, an else if chain tests conditions in order and runs only the first block whose condition is true, so the order of the tests matters. Arranging bands in order, lowest first or highest first, means each later condition needs only one bound, because earlier branches have already removed the other values. The final else catches anything not matched, which is useful for out-of-range or unexpected data. Nested selection puts one if statement inside another, so the inner decision is only made when the outer condition has a particular result; for example, a hospital fridge monitor might first ask whether a reading is within the safe range and only then decide whether an unsafe reading is too cold or too warm. Conditions are built using relational operators and combined with && and ||, as covered in AP19. Clear indentation shows which statements belong to which branch, and deep nesting should be avoided where a chain or combined condition reads more clearly.

The switch statement

A switch statement compares the value of one variable or expression with a list of constant case labels and jumps to the matching case. In C#, each case section must end with break (or another jump such as return) so that control does not fall through into the next case; several labels can be stacked above one section so that they share the same code, for example treating 6 and 7 as weekend days. The default section runs when no case matches, acting like the final else in a chain. A switch is clearer and less repetitive than a long else if chain when a single value is being matched against fixed options, such as menu choices, day numbers or category codes, and it makes the set of options easy to see at a glance. Its limitation is that the classic form tests only for equality with exact values, so it cannot directly express ranges such as age below 16; an if... else if chain is needed for those. Newer versions of C# do allow relational patterns in a switch, but the AS course expects the traditional form. In pseudocode, the equivalent is usually written as CASE ... OF with a list of options and an OTHERWISE clause.

Count-controlled loops

A count-controlled loop is used when the number of repetitions is known before the loop begins, either because it is fixed, such as five working days, or because it has already been entered, such as the number of people in a group. In C# this is a for loop, whose header has three parts: initialisation of an integer loop counter, a condition checked before each pass, and an update applied after each pass, usually i++ to add one. If the condition is true, the body runs again; once it is false, the loop ends and execution continues after it. The counter can be used inside the body, for example to number prompts, and it can count down (i--) or in steps (i += 3). Off-by-one errors are common, so check the boundaries carefully: for (int i = 1; i <= 5; i++) runs five times, as does for (int i = 0; i < 5; i++), but i < 5 starting from 1 runs only four times. In pseudocode, a count-controlled loop is written as FOR i ← 1 TO 5 ... NEXT i, or in plain English as REPEAT 5 TIMES. The foreach loop, which visits every element of an array or list, is also count-controlled and is covered with data structures in AP22.

Condition-controlled loops and sentinels

A condition-controlled loop repeats while, or until, a condition is met, so it is used when the number of repetitions is not known in advance. A while loop is a pre-condition loop: the condition is tested before each pass, so if it is false at the start the body never runs at all. This suits processing a list of values that ends with a sentinel, a dummy value such as -1 that marks the end of the data and is not itself processed. The standard pattern reads the first value before the loop and reads the next value as the last statement in the body. If the sentinel is entered first, the loop correctly processes nothing, but any later calculation such as an average must then guard against dividing by zero. A do...while loop is a post-condition loop: the body runs first and the condition is tested at the end, so it always runs at least once, which suits menus and input validation where the user must be asked at least once. The equivalent pseudocode REPEAT ... UNTIL stops when its condition becomes true, which is the opposite of do...while. If nothing inside the body can change the condition, the loop never ends: an infinite loop, which is a logic error.

Nested structures and tracing

Structures can be placed inside one another. With nested loops, the inner loop runs completely for every single pass of the outer loop, so the total number of times the inner body runs is the product of the two loop counts: an outer loop of 3 and an inner loop of 4 gives 12. Nested loops naturally process two-dimensional data such as seating plans, timetables, multiplication grids and the 2D arrays met in AP22. Real programs mix all three constructs, for example a validation loop to read a group size, then a for loop to process each member, with else if selection inside to place each person in a price band. To check such code, use a trace table: a column for each variable and condition, and a new row each time a value changes. Work through the code one statement at a time exactly as the computer would, rather than guessing the pattern. Trace tables reveal logic errors such as off-by-one boundaries, a sentinel being added to the total, or a counter that is never updated. Meaningful names (row, col, count) and consistent indentation make nested code far easier to read, trace and debug.

Key terms from the mark schemes

Sequence
Executing instructions one after another in the order in which they are written.

Selection
A construct in which a condition determines which of two or more paths through the program is followed.
Credited: decision / branching

Iteration
Repeating a set of instructions, either a set number of times or while/until a condition is met.
`Credited: repetition / looping`

Condition-controlled loop
A loop that repeats while or until a condition is met, so the number of repetitions need not be known in advance.
`Credited: indefinite iteration`

Nested selection
An if statement placed inside another if or else block, so the inner decision is only made when the outer condition has a particular result.
`Credited: nested if`

Pre-condition loop
A loop that tests its condition before each pass, so the body may run zero times (while).

Infinite loop
A loop whose terminating condition is never met, usually because the control variable is not changed in the body.

Count-controlled loop
A loop that repeats a fixed number of times known before it starts, e.g. a for loop.
`Credited: definite iteration`

Nested loop
A loop placed inside the body of another loop; the inner loop completes fully for each pass of the outer loop.

Sentinel value
A dummy value entered to mark the end of the data, which is not processed as part of the data.
`Credited: rogue value / terminator`

Post-condition loop
A loop that tests its condition after each pass, so the body always runs at least once (do...while / REPEAT... UNTIL).

C# example

`Sequence: order matters`

```
Console.Write("Adult tickets: ");
int adults = int.Parse(Console.ReadLine());
Console.Write("Child tickets: ");
int children = int.Parse(Console.ReadLine());
double total = adults * 9.50 + children * 6.00;
total = total * 0.9;                        // 10% online discount
Console.WriteLine($"Total: £{total:F2}");
// Input 2 and 3  ->  Total: £33.30
```

Exam focus

Questions often give an algorithm and ask you to explain sequence, selection and iteration with reference to it; one mark is typically for the definition and one for identifying where it appears, so quote the relevant line, for example 'the IF total >= 500 statement only applies the discount when the condition is true'. For the difference between count-controlled and condition-controlled loops, state that a count-controlled loop repeats a number of times known before it starts, while a condition-controlled loop repeats until or while a condition is met, and give an example of each. For while versus do...while, state where the condition is tested and the consequence: zero runs are possible with while; at least one with do...while. When asked to justify a loop choice, link it to whether the number of repetitions is known. Code-writing questions credit correct logic, sensible variable names and correct boundaries, so check < versus <= carefully; mark schemes often accept equivalent code in any sensible form. In trace questions, write out every value in order rather than jumping to the answer. Common mistakes include processing the sentinel value, confusing UNTIL with WHILE conditions and forgetting that only the first true branch of an else if chain runs.

? Check yourself

1. State the three basic programming constructs. [3]
2. A Ballymena sports shop's loyalty app uses this algorithm: points ← 0; FOR purchase ← 1 TO 5: INPUT spend; points ← points + spend DIV 10; NEXT purchase; IF points >= 50 THEN OUTPUT "Voucher earned" ENDIF; OUTPUT points. By referring to the algorithm, explain the terms (a) sequence (b) selection (c) iteration. [6]
3. Describe the difference between a while loop and a do...while loop. [2]

Answers: 1. Sequence (1) / selection (1) / iteration or repetition (1) 2. (a) Sequence: instructions are executed one after another in the order written (1) e.g. points is set to 0 before the loop, and the voucher check and final OUTPUT points happen only after all five spends have been processed (1) (b) Selection: a condition decides which path is followed / whether a block runs (1) e.g. "Voucher earned" is output only IF points >= 50 (1) (c) Iteration: a set of instructions is repeated (1) e.g. INPUT spend and the points update are repeated once for each of the 5 purchases by the FOR loop (1) 3. A while loop tests the condition at the start so it may run zero times (1) / a do...while loop tests the condition at the end so it always runs at least once (1)